RAG Cheat Sheet — Discrete Concept Compartments

AI Support Agent (Better Sleep) · Print this. Study one box at a time.

Big idea (one sentence)

Don't ask the AI to remember your catalog. Retrieve approved facts first, then ask the AI to write the customer reply using only those facts. That loop is RAG.

R — Retrieve

Plain: find the right facts.

In this project:
- knowledge.search_knowledge()
- orders.get_order_help_facts()

Output = Snippets / OrderHelpFacts (truth objects, not pretty prose)

Later: embeddings + vector DB when docs get large.

A — Augment

Plain: attach facts to the ask.

In this project:
- Build a factual template
- Put FACTS in the prompt
- Pass customer question
- Pass support lane

Augment = give the model context it did not train on (your docs / order record).

G — Generate

Plain: write the reply.

In this project:
- responses._warm_rewrite()
- ai_client.complete()

AI rephrases facts warmly.
If AI fails → template stays.

Model = writer, not warehouse.

Compartment: RAG vs guessing

Pure LLM alone:
answers from training memory → can invent

RAG:
answers from your retrieved facts → auditable

Fine-tuning (different tool):
teaches style/format; still not a live catalog

We use RAG so product/policy changes = edit docs, not retrain a model.

Compartment: our file roles

knowledge.py / orders.py	→ RETRIEVE facts
responses.py	→ AUGMENT + GENERATE wording
ai_client.py	→ talks to OpenAI (connection only)
guardrails.py	→ hard safety before/after
support.py	→ which lane am I in?
main.py	→ wires the pipeline

Remember: change docs/data to change truth; change responses/ai_client to change voice/provider.

Compartment: vocabulary (ruminate here)

Grounding	Tying answers to approved sources so claims can be checked.	**Hallucination**	Model invents a confident-sounding claim not in the facts.
Snippet	One retrieved section from a knowledge doc (our fact unit).	**Faithfulness**	Did the reply stick to retrieved facts? (eval metric later)
Embedding	A numeric 'meaning fingerprint' of text for similarity search (future).	**Warm rewrite**	Our current G step: AI rephrases a factual template more warmly.
Vector store	Database of embeddings (Chroma/pgvector) for fast similarity lookup (future).	**System prompt**	Standing instructions to the model (persona + hard rules).

Roadmap: keyword retrieve (now) → better filters → embeddings/vector DB → hybrid → faithfulness eval → multi-turn state

RAG in this repo — one worked example

Follow the arrows. Each compartment is one mental step.

1. Customer asks

`"Where is order ORD-1001?"`

main.py collects the question. Guardrails check input first.

2. Classify lane

`support.classify_request → Order Help`

Lane is context we pass forward (prefer the right tools).

3. Retrieve facts

`orders.get_order_help_facts(...)`

Truth: status=shipped, tracking=1Z..., ETA=2026-07-24. No prose yet.

4. Augment

`responses builds a factual template + prompt`

FACTS block + question + lane go to the model. System prompt bans inventing.

5. Generate

`ai_client.complete(...) warm rewrite`

Friendly wording, same numbers. If API fails, template is shown as-is.

6. Safety net

`guardrails.apply_output_guardrails(...)`

Blocks refund promises, diagnosis-ish wording, unexpected emails.

Ruminate prompts (pick one)

- If I deleted ai_client.py, could the agent still tell the truth? (Yes — templates.)
- If I edited data/knowledge/products.md, would answers change without retraining? (Yes — RAG.)
- If the model 'sounds right' but the tracking number isn't in OrderHelpFacts, what failed? (Grounding.)